

Construire des arbres de décision

Classificateurs, if-then-else, et apprentissage supervisé

Traduit et adapté depuis Bootstrap World (Fall 2026)

Adapté au contexte francophone — colonnes et espèces en français

Licence Creative Commons 4.0 International

*Ces supports ont été développés en partie grâce au soutien de la National Science Foundation
(bourses 1042210, 1535276, 1648684, 1738598, 2031479 et 1501927).*

Table des matières

1	Prédire des catégories	3
1.1	Lancement	3
1.2	Exploration : « Je vois... Classifier ! »	4
1.2.1	Vocabulaire des arbres de décision	8
1.3	Exploration : le jeu de données des animaux	8
1.4	Synthèse	10
2	Construire des arbres de décision en Pyret	12
2.1	Lancement	12
2.2	Visualiser et évaluer l'arbre	15
2.3	Choisir une division	17
2.4	Synthèse	19
2.4.1	Aller plus loin : publicité ciblée et filtrage de spam	19

Prédire des catégories

Objectif de la section

Les élèves découvrent les *classificateurs* et construisent des **arbres de décision** en jouant à une version adaptée du jeu « Je vois, je vois ». Ils apprennent ce qu'est un arbre de décision, qu'il existe plusieurs arbres possibles pour un même jeu de données, et que la qualité d'un arbre dépend de la façon dont il sépare les données.

Lancement

Considérez chacune des trois situations suivantes :

- Vous et vos ami·es passez un quiz en ligne qui tente de deviner votre coiffure.
- Deux personnes qui travaillent pour la *même entreprise* et gagnent le *même salaire* demandent un prêt à la banque en remplissant le *même formulaire en ligne*. L'une obtient le prêt, l'autre non.
- Un·e journaliste affiche une carte avec certains États colorés en bleu et d'autres en rouge. Iel explique que les prédictions de la carte pour l'élection à venir ont été établies à partir de données historiques du recensement.
- Une caméra à la banque prend une photo de votre visage, dans l'espoir de la faire correspondre à votre identité.

Réflexion

Qu'est-ce que tous ces exemples ont en commun ?

En surface, ces situations ne semblent pas avoir grand-chose en commun. Mais tous sont des exemples de **classificateurs** — une sorte de fonction qui :

1. reçoit une entrée ;
2. pose des questions sur cette entrée ;
3. utilise les réponses pour produire un résultat prédit à partir d'une liste de catégories.

Réflexion

- Quelles données pensez-vous être utilisées pour **deviner votre coiffure** ?
 - Historique de visionnage, recherches, abonnements aux chaînes, likes, dislikes, activité Google, retours « non intéressé ».
- Quelles données pour **l'approbation d'un prêt** ?
 - Dettes, actifs, historique de paiement, nombre de comptes ouverts, comptes en retard, revenus, charges, découverts des 90 derniers jours.
- Quelles données pour **prédire une élection** ?
 - Données du recensement incluant race, genre, niveau de revenu, affiliation politique, âge des votant·es, ainsi que les résultats des années précédentes.

Point clé

Les classificateurs sont des fonctions qui associent une entrée à un ensemble possible de cibles.

Exemple	Données (connues)	d'entrée	Catégories (inconnues)
Quiz coiffure	Sexe, âge, race, musique, code postal...		Queue de cheval vs. coupe courte vs. dreadlocks vs. crew cut vs. mohawk...
Prêt bancaire	Revenu, emploi, âge, score de crédit...		Approuvé vs. Non approuvé
Élections	Votes passés, sondages récents...		Bleu vs. Rouge
Reconnaissance faciale	Votre visage		Toute personne enregistrée

Par exemple, le quiz de coiffure *probablement* éliminera « queue de cheval » si la personne qui le passe est « masculine », mais il ne se trompera pas toujours : après tout, certains hommes portent une queue de cheval ! Le travail d'un classificateur est de deviner l'**option la plus probable** à partir des données d'entraînement.

La plupart des modèles se trompent. Certains sont utiles.

Exploration : « Je vois... Classifier ! »

Les classificateurs utilisent ce qu'on appelle un **arbre de décision** : ils posent des questions pour réduire les résultats possibles jusqu'à atteindre un seul résultat prédit.

Si vous avez déjà joué à « Je vois, je vois avec mon petit œil », vous avez déjà construit un arbre de décision ! Les règles sont simples :

- Un·e joueur·euse pense à quelque chose que tout le monde peut voir.
- Iel donne un indice : « Je vois... quelque chose de bleu ! »
- Les autres joueur·euses posent des questions fermées (oui/non) pour deviner.

Activité

Nous allons jouer à une version « Je vois... **Classifier!** » :

- Tout le monde se lève à sa place.
- L'enseignant·e choisit un·e élève comme cible, sans le dire à personne.
- Tour à tour, les élèves posent des questions fermées.
- Vous pouvez poser des questions *catégorielles* : « porte-t-il des lunettes ? »
 - Si la réponse est « oui », seul·es les élèves portant des lunettes restent debout.
 - Si la réponse est « non », seul·es ceux·celles qui *ne portent pas* de lunettes restent.
- Vous pouvez poser des questions *quantitatives* : « mesure-t-il plus de 1 m 70 ? »
 - Si « oui », seul·es les élèves de plus de 1 m 70 restent.
 - Si « non », seul·es ceux·celles de 1 m 70 ou moins restent.

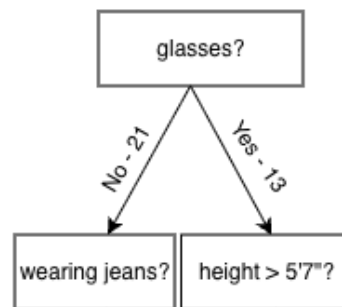


FIGURE 1 – Une partie d'un arbre de décision montrant le nombre d'élèves qui portent ou non des lunettes.

Notes pour l'enseignant·e

Pendant que la classe essaie de deviner la cible, dessinez au tableau la progression sous forme d'organigramme, avec des arêtes « oui » et « non » ramifiées vers la gauche et la droite. Sur chaque arête, notez le nombre d'élèves pour qui la réponse est « oui » et « non ». Continuez jusqu'à ce que la classe ait trouvé la cible.

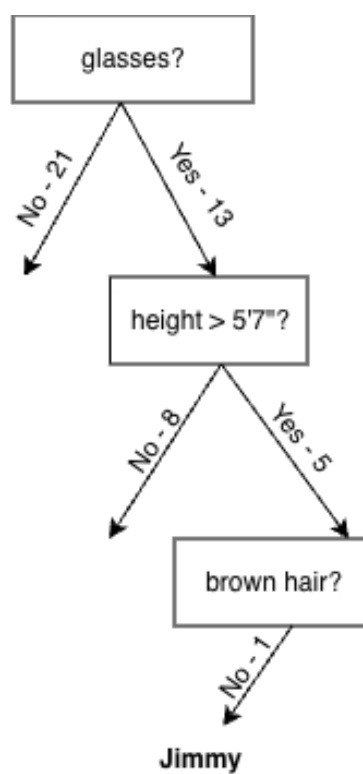


FIGURE 2 – Un arbre de décision représentant la recherche d'un-e élève en classe, montrant les nombres d'élèves qui portent ou non des lunettes, qui ont ou non les cheveux roux, et qui sont ou non des filles. Ici, la cible identifiée est « Jimmy ».

Réflexion

- **Combien de questions** a-t-il fallu pour trouver la cible ?
- **Si vous aviez posé « s'appelle-t-iel XYZ ? » en boucle jusqu'à trouver la cible ?** Auriez-vous fini par trouver ?
 - Oui, mais cela aurait pris *beaucoup plus longtemps* ! Poser des questions sur des groupes a permis d'éliminer des cibles possibles et de réduire les choix plus rapidement.

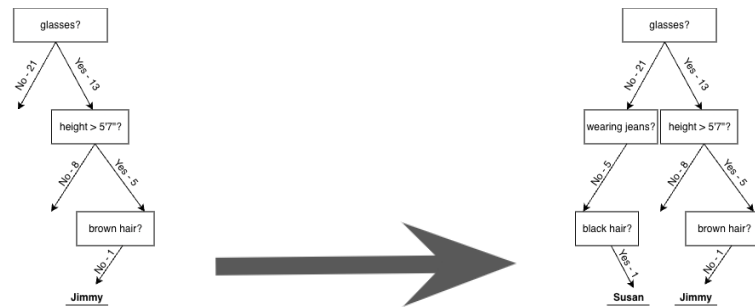
Activité

Rejouons quelques manches. Cette fois, cependant, nous ne pouvons *qu'ajouter* à notre arbre : pour tout le reste, il faut poser les mêmes questions dans le même ordre.

Notes pour l'enseignant·e

Rejouez quelques manches. Essayez de choisir des élèves qui forceront la classe à **ajouter à leur arbre**. Pour chaque élève classifié, demandez à la classe combien de questions ont été nécessaires.

Encouragez-les à poser un mélange de questions catégorielles et quantitatives. Si les élèves commencent à se plaindre d'être « coincé-es » avec leurs premières questions, c'est bon signe ! Demandez-leur ce qu'ils changeraient et pourquoi – un excellent moyen de faire émerger leur intuition sur ce qui fait un « bon » arbre de décision.



Réflexion

- Dans ce jeu, quelles étaient nos « cibles » ?
 - Les élèves de la classe.
- Combien de cibles possibles avions-nous ?
 - Le nombre d'élèves dans la classe.
- Si vous pouviez recommencer, comment changeriez-vous cet arbre de décision ?
 - Grande question ! Laissez les élèves discuter.
- Si notre première question était « porte-t-iel des lunettes ? » et qu'un seul élève porte des lunettes : si cet élève est la cible, combien de questions supplémentaires seront nécessaires ?
 - Aucune ! Si la réponse est « oui », on sait immédiatement qui c'est.
- Serait-ce la *meilleure* première question si on devait jouer souvent ?
 - Non, car si la réponse est « non » on n'a pas réduit grand chose.
- Si *presque tous* les élèves mesurent 1 m 70 ou plus, « mesure-t-iel plus de 1 m 70 ? » serait-il une bonne première question ?
 - Non, car la question ne sépare pas le groupe d'une façon qui reflète la variation réelle des données.
- Quand est-ce que « plus de 1 m 70 ? » serait une *bonne* question ?
 - Si elle divise la classe aussi près que possible « en deux ».

Vocabulaire des arbres de décision

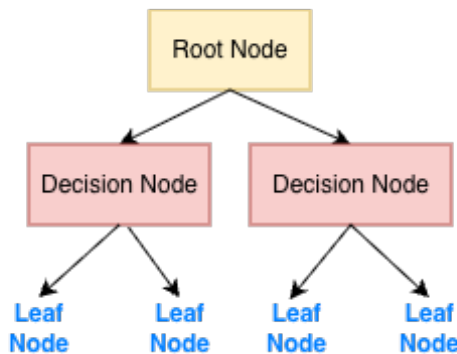


FIGURE 3 – Un arbre de décision utilisé pour introduire le vocabulaire : nœud, nœud racine, nœud feuille.

Point clé

Vocabulaire des arbres de décision

- Un **nœud de décision** (*decision node*) divise un jeu de données selon les valeurs d'une de ses colonnes. C'est une « question ».
- Le tout premier nœud (en haut) est appelé le **nœud racine** (*root node*). Il représente le jeu de données entier.
- Chaque division crée davantage de branches et de nœuds, représentant des sous-ensembles de données selon une nouvelle « question » sur une colonne.
- Un **nœud feuille** (*leaf node*) contient le résultat prédit, et n'a pas d'autres branches. Les résultats proviennent d'une colonne désignée du jeu de données.

Notes pour l'enseignant·e

Nous nous limitons aux divisions qui ne créent que deux branches, mais certains arbres de décision utilisent des divisions en 3, 10 ou plus. Les informaticien·nes appellent la séquence de questions depuis la racine jusqu'à un résultat un « chemin » (*path*). Discutez de l'arbre de décision construit pendant le jeu pour aider les élèves à identifier le nœud racine, les nœuds de décision, les nœuds feuilles et les chemins.

Exploration : le jeu de données des animaux

Nous allons jouer à une autre manche de « Je vois... Classifier ! », mais cette fois avec un vrai jeu de données d'animaux.

nom	espece	sexe	poids	queue	mammifere	nage
Firepaw	chat	male	9.50	TRUE	TRUE	FALSE
Leafpool	chat	femelle	10.60	TRUE	TRUE	FALSE
Shadestar	chat	femelle	9.00	TRUE	TRUE	TRUE
Bramble	chat	femelle	4.10	TRUE	TRUE	FALSE
Brick	chat	male	8.50	TRUE	TRUE	FALSE
Frisky	chien	femelle	9.60	TRUE	TRUE	TRUE
Socks	chien	femelle	10.00	TRUE	TRUE	TRUE
Stripe	chien	male	11.10	TRUE	TRUE	TRUE
Snickers	chien	male	10.30	TRUE	TRUE	TRUE
Bruni	lezard	femelle	8.20	TRUE	FALSE	TRUE
Jesús	lezard	male	3.50	TRUE	FALSE	FALSE
Gary	escargot	hermaphrodite	0.02	FALSE	FALSE	FALSE
Turbo	escargot	hermaphrodite	0.03	FALSE	FALSE	FALSE
Charlotte	tarentule	male	0.35	FALSE	FALSE	FALSE
Webs	tarentule	femelle	0.40	FALSE	FALSE	FALSE

Réflexion

- Que **remarquez**-vous sur les animaux dans ce jeu de données ?
 - Les réponses varieront : Shadestar est un chat qui nage ! Charlotte est une tarentule mâle ! Les escargots sont très légers !
- Quels **types de données** voyez-vous dans cette table ?
 - Strings (nom, espece, sexe), Numbers (poids), Booleans (queue, mammifere, nage).

Point clé

Un arbre de décision est une façon de **modéliser la structure cachée** dans les données. Contrairement aux humains, qui peuvent inventer leurs propres questions, les algorithmes d'apprentissage automatique *génèrent* des arbres de décision à partir de jeux de données d'entraînement, en utilisant les en-têtes de colonnes comme questions. Chaque nœud dans l'arbre divise le jeu de données, nous permettant de faire la meilleure prédiction possible sur un résultat.

Construire un arbre de décision est une forme d'**apprentissage supervisé**, car les données d'entraînement contiennent déjà les résultats souhaités (dans une colonne), et l'algorithme apprend une fonction qui fait correspondre les entrées aux résultats.

Activité

- L'enseignant·e choisit un animal du jeu de données.
- Votre travail est de deviner son *espèce* (chat, chien, lézard, escargot, ou tarentule).
- Vous pouvez poser toutes les questions qui peuvent être répondues en utilisant les colonnes de la table.

En petits groupes, jouez quelques manches ! Certaines questions sont-elles meilleures que d'autres ?

Au mur ou sur une feuille de brouillon : quel est, selon vous, le **meilleur nœud racine** pour ce jeu de données ? (C'est-à-dire : quelle est la question la plus utile pour commencer ?) Pouvez-vous expliquer pourquoi vous avez choisi cette question ?

Notes pour l'enseignant·e

Encouragez les élèves à proposer à la fois des questions catégorielles et quantitatives, et demandez-leur sans cesse d'expliquer *pourquoi* ils ont choisi cette division.

Pendant qu'ils explorent, dessinez l'arbre de décision au fur et à mesure. Le format Desmos « Choosing a Root Node » peut accompagner cette exploration.

Synthèse

Réflexion

- En quoi *générer des questions* à partir d'un jeu de données était-il *similaire* au jeu « Je vois... Classifier ! » précédent ?
 - Certaines questions faisaient un meilleur travail de séparation des données que d'autres.
- En quoi était-ce *différent* ?
 - Il y avait des questions que nous ne pouvions pas poser, car il n'y avait pas de colonne pour elles !

Point clé

Vous avez appris que certains arbres de décision sont meilleurs que d'autres. Par exemple, la *fréquence de chaque catégorie* affecte le « meilleur nœud » (« si tout le monde porte des lunettes... »).

Lors du choix d'une division pour notre nœud racine, notre but est de poser une question qui sépare les données en *tas les plus nets et organisés possibles*. Par exemple, il serait idéal d'avoir tous les chats d'un côté et tous les chiens de l'autre. Nous ne pouvons pas toujours obtenir une séparation parfaite, mais nous voulons nous en approcher autant que possible.

Réflexion

- **Comment sait-on qu’une question est bonne pour commencer ?**
 - Les bonnes questions aident à éliminer beaucoup de catégories restantes, que la réponse soit oui *ou* non.
- **Que voulons-nous être vrai pour chaque question que nous posons ?**
 - Qu’elle sépare la liste des cibles possibles le plus proprement possible, tout en éliminant autant d’options que possible.
- **Comment utilise-t-on un arbre de décision pour faire des prédictions ?**
 - On pose la question à la racine de l’arbre, puis on suit les branches pour poser des questions supplémentaires jusqu’à atteindre une catégorie à la fin d’un chemin.
- Si nous jouions à « Classifier », mais en devinant quel **pays** je pense : quelle serait une bonne première question ?
 - Est-ce un pays de l’hémisphère occidental ?
 - Est-ce un pays où l’on parle espagnol ?

Construire des arbres de décision en Pyret

Objectif de la section

Les élèves apprennent à écrire une fonction *classificateur* en Pyret pour prédire l'espèce d'un animal, en utilisant des expressions *if/else* pour poser des questions sur une ligne (*row*) de la table. Ils utilisent *image-dot-plot* et *classify* pour visualiser et évaluer leur arbre.

Lancement

Apprenons à écrire une fonction classificateur pour prédire l'espèce en Pyret !

Point clé

Rappelez-vous :

- Une fonction consomme une entrée et produit une sortie.
- Pour la *même entrée*, une fonction doit produire le *même résultat* — à chaque fois.
- Cela signifie que les classificateurs sont *juste des fonctions* ! Notre classificateur classera le même animal de la même façon, à chaque fois.
 - S'il prédit que *f irepaw* est un chat, il prédira *toujours* cela.
 - S'il prédit que *f irepaw* est un escargot, il prédira *toujours* cela.

Nous devons choisir une question, puis écrire une règle.

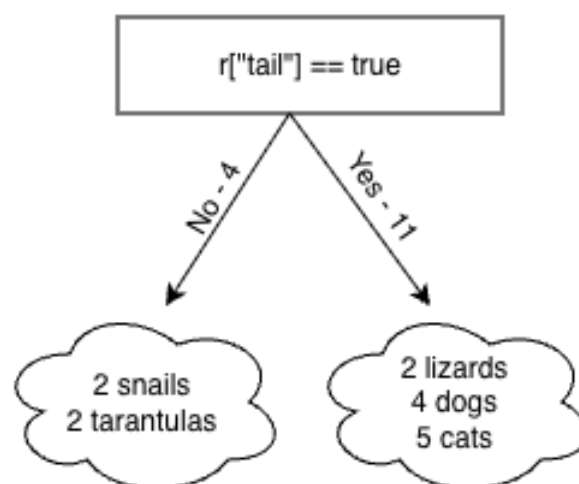


FIGURE 4 – Un arbre de décision représentant une recherche d'espèce d'animal. Le nœud racine divise selon la présence d'une queue. La branche « non » contient 2 tarentules et 2 escargots, tandis que la branche « oui » contient 2 lézards, 4 chiens et 5 chats.

Réflexion

Supposons que notre arbre n'ait qu'un seul niveau : on demande si l'animal a une queue, puis on prend notre meilleure estimation.

Parmi les 15 animaux de notre jeu de données d'entraînement, 11 ont une queue et 4 n'en ont pas.

- Quelle espèce notre règle devrait-elle prédire pour le côté « oui » de l'arbre ?
 - « chat » – c'est l'espèce la plus commune.
- Quelle espèce pour le côté « non » de l'arbre ?
 - « escargot » – escargot et tarentule sont également communs, mais on doit en choisir un.

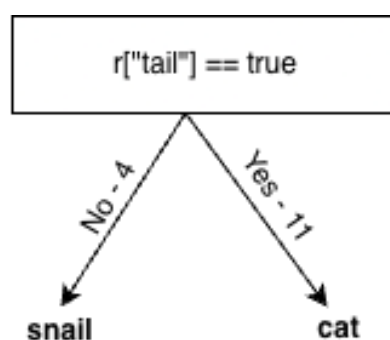


FIGURE 5 – Un arbre de décision qui prédit l'espèce d'un animal. Le nœud racine divise selon la présence d'une queue. La branche « non » prédit escargot, la branche « oui » prédit chat.

Point clé

Un arbre de décision est un **algorithme avec perte** (*lossy algorithm*). Il réduit la complexité et écarte la variance des données, en cherchant des « règles générales » qui s'appliquent à *la plupart* – mais pas à la totalité – des données.

- La branche « oui » contient plus de chats que toute autre chose (5 sur 11). Même si les chats sont l'espèce la plus probable pour les animaux à queue, notre classificateur ne sera juste qu'environ 45 % du temps !
- La branche « non » est également répartie entre escargots et tarentules. Quelle que soit l'espèce prédite par notre règle, le classificateur ne sera juste que 50 % du temps !

Activité

- Ouvrez le **fichier de démarrage Arbre de Décision** (Arbre-Decision-Starter.arr) et cliquez sur « Run ».
- Remarquez que les premières lignes de la Zone de Définitions chargent les données et définissent 2 tables, appelées `training` et `testing`.
- Tapez `training` dans la Zone d'Interactions pour voir la table avec laquelle nous allons travailler pour l'instant.

Voici comment nous définirions notre `classifieur-queue` en Pyret :

Code Pyret

```
# classifieur-queue :: Row -> String
# consomme un animal, et prédit l'espèce
fun classifieur-queue(r):
  if r["queue"] == true:
    "chat"
  else:
    "escargot"
  end
end
```

Réflexion

Vous avez probablement remarqué que :

- Comme toute autre fonction, `classifieur-queue` a un **contrat** ! Elle consomme une *Row* (un animal) et produit une *String* (l'espèce prédite).
- `classifieur-queue` utilise un *accesseur de ligne* pour accéder à l'information dans la colonne "queue".
- Cette fonction utilise un code que nous n'avons pas vu avant, comme `if` et `else`.

Activité

1. Que fait cette partie de la définition de fonction ? `if r["queue"] == true: "chat"`
 - Elle vérifie si la valeur dans la colonne "queue" est `true`, et si oui, elle classe l'animal comme un « chat ».
2. Que fait `else: "escargot"` ?
 - Si la valeur n'est pas `true`, elle classe l'animal comme un « escargot ».
3. Que renvoie `classifieur-queue(firepaw)` ?
 - « chat », car Firepaw a une queue.

Notes pour l'enseignant·e

Vous trouverez peut-être redondant de vérifier `r["queue"] == true` puisque la valeur de la cellule est elle-même `true` ou `false`; et vous avez raison! Le programme fonctionnerait aussi bien en lisant juste `r["queue"]`. Mais utiliser `r["queue"] == true` reste cohérent avec la façon dont on utilise les accesseurs de ligne avec les colonnes non booléennes (e.g. `r["espece"] == "chat"`). L'expérience montre que certain·es élèves s'en sortent mieux si tout reste cohérent.

Visualiser et évaluer l'arbre

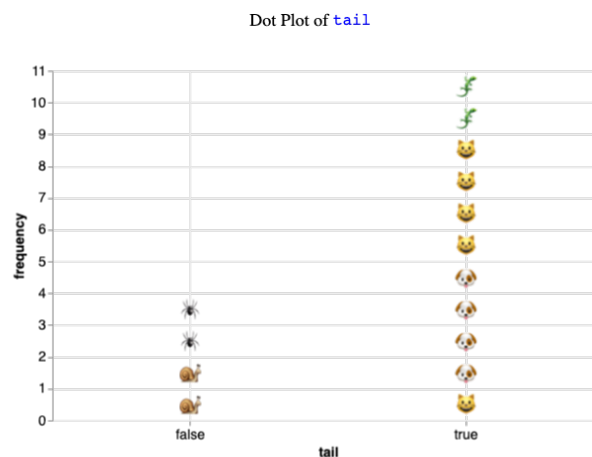


FIGURE 6 – Un *dot plot* montrant la distribution des valeurs de queue (true/false) sur un petit jeu de données d'animaux. Chaque point est un emoji de l'espèce de l'animal.

Vous venez de rencontrer deux nouvelles fonctions Pyret qui vont nous aider à donner du sens aux données pendant que nous construisons nos arbres.

Point clé

Deux nouveaux outils

- La fonction `image-dot-plot` utilise des emojis pour chaque animal au lieu de points, pour nous aider à voir comment les espèces sont distribuées de part et d'autre de la division de notre classificateur.
- La fonction `classify` ajoute les prédictions de notre classificateur à la table et calcule si elles sont correctes ou non.

```
# classify :: (Table t, String colonne-reponse,
              (Row -> String) classifieur) -> Table
```

```
# image-dot-plot :: (Table t, String colonne, (Row
-> Image) image-fn) -> Image
```

Utilisez l'un de nos nouveaux outils pour répondre aux questions suivantes sur notre arbre classifieur-queue :

Code Pyret

```
image-dot-plot(training, "queue", animal-img)
classify(training, "espece", classifieur-queue)
```

Réflexion

- Pour combien d'animaux **avec queue** dans le jeu de données d'entraînement, notre arbre à 1 niveau fera-t-il la *bonne* prédiction ?
 - 5 — tous les chats.
- Pour combien d'animaux **sans queue** ?
 - 2 — tous les escargots.
- Pourquoi l'arbre se trompe-t-il pour les chiens et les lézards ?
 - Parce qu'ils ont tous une queue, et l'arbre devine toujours « chat » pour les animaux à queue.
- Pourquoi se trompe-t-il pour les tarentules ?
 - Parce qu'elles n'ont pas de queue, donc l'arbre devine toujours « escargot » pour elles.
- Est-ce que l'utilisation de `queue` comme nœud racine a fait du bon travail pour classer les animaux ?
 - Pas vraiment. Seulement la moitié de ses prédictions étaient correctes.

Choisir une division

Activité

Utilisez `image-dot-plot` pour visualiser la distribution des animaux selon `mammifere`, `queue`, `nage` et `sexe`, puis complétez le tableau ci-dessous pour identifier les meilleures divisions *catégorielles* :

Diviser sur	Feuille « non » (espèce majori- taire)	Feuille « oui » (espèce majori- taire)	# Correct
<code>r["queue"] == true</code>	« escargot » ou « tarentule » (2 sur 4)	« chat » (5 sur 11)	7
<code>r["mammifere"] == true</code>	« escargot » ou « tarentule » (2 sur 4)	« chat » (5 sur 9)	7
<code>r["nage"] == true</code>	(à compléter)	(à compléter)	
<code>r["sexe"] == "male"</code>	(à compléter)	(à compléter)	
<code>r["sexe"] == "femelle"</code>	(à compléter)	(à compléter)	

- Pourquoi n'aurait-il pas de sens d'avoir une ligne pour `r["mammifere"] == false`?

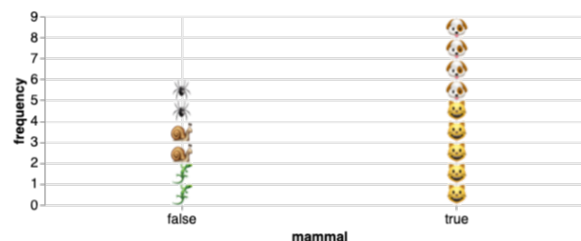


FIGURE 7 – Dot plot montrant la distribution de la colonne `mammifere` (true/false) sur le jeu d'animaux. Chaque point est un emoji de l'espèce.

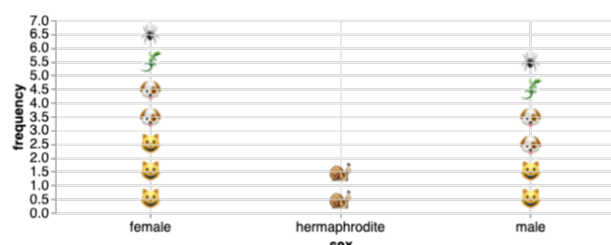


FIGURE 8 – Dot plot montrant la distribution du `sexe` des animaux. Chaque point est un emoji de l'espèce.

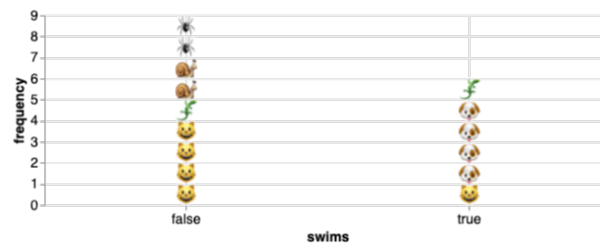


FIGURE 9 – Dot plot montrant la distribution de la colonne `swims` (true/false) sur le jeu d'animaux. Chaque point est un emoji de l'espèce.

Activité

Maintenant, les divisions *quantitatives* : utilisez `image-dot-plot` pour visualiser la distribution des animaux selon le `poids`.

Diviser sur	Feuille « non »	Feuille « oui »	# Correct
<code>r["poids"] > 2</code>	« escargot » ou « tarantule » (2 sur 4)	« chat » (5 sur 11)	7
<code>r["poids"] > 6</code>	(à compléter)	(à compléter)	
<code>r["poids"] > 9.5</code>	(à compléter)	(à compléter)	

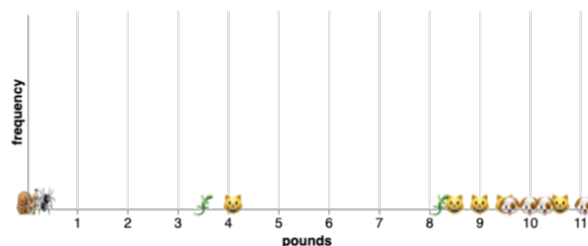


FIGURE 10 – Dot plot montrant la distribution des `poids` sur le jeu d'animaux. Chaque point est un emoji de l'espèce.

Réflexion

- **Comment choisit-on un nœud racine qui fera la meilleure division possible ?**
 - On choisit une division qui atteint le plus de prédictions correctes pour les données d'entraînement.
- **Comment regarder le dot plot nous aide-t-il à choisir un nœud racine ?**
 - Si un cluster contient majoritairement une seule espèce, une division tombant dans les espaces entre ces clusters peut être une excellente candidate !
- **Les valeurs aberrantes aident-elles à choisir un nœud racine ?**
 - Les valeurs aberrantes ne représentent probablement pas la majorité de leur espèce, donc on pourrait ne pas vouloir les prioriser dans nos divisions.

Notes pour l'enseignant·e

Tous les arbres que nous utilisons ici sont *binaires* (chaque nœud n'a que deux chemins). Certains arbres ont des nœuds qui se divisent en 3, 10 ou plus.

Si on divise sur `r["nage"] == true`, on obtient :

- oui : 4 chats, 1 lézard, 2 escargots, 2 tarentules.
- non : 4 chiens, 1 chat, 1 lézard.

Activité

Quels nœuds de décision devrions-nous utiliser ensuite pour la branche « oui » ? Pourquoi ?

Les réponses pourraient inclure :

- `r["mammifere"] == true` séparerait efficacement les chats de tous les autres.
- `r["poids"] < 4` atteindrait la même chose.
- `r["queue"] == true` séparerait les chats et lézards des escargots et tarentules.

C'est le *même* problème que celui que nous avons résolu pour le nœud racine : notre but est de poser une question qui trie les données en tas les plus nets et organisés possibles. Complétez le reste de l'algorithme en dessinant votre arbre à deux niveaux et en écrivant `mon-classifieur2`.

Synthèse

Réflexion

- **Expliquez comment l'arbre de décision et le jeu de données d'entraînement se correspondent.**
 - Les en-têtes de colonnes de la table sont les questions qui seront posées dans les nœuds de décision de l'arbre.
 - Les données dans chaque colonne sont les réponses à ces questions, qui deviennent les étiquettes des arêtes de l'arbre (« oui » et « non » pour ce jeu de données).
 - Les catégories de sortie de la table sont les nœuds feuilles de l'arbre.

Point clé

Un modèle est un résumé mathématique d'un jeu de données.

Que résume un arbre de décision à propos d'un jeu de données ? Que laisse-t-il de côté ?

Aller plus loin : publicité ciblée et filtrage de spam

Considérez ces deux jeux de données. Pourriez-vous écrire un classificateur pour chacun ?

Publicité ciblée :

nom	âge	historique d'achat	intérêt pour le jeu	achète le jeu
Jan	ados	client·e existant·e	faux	faux
Noah	trentaines	client·e existant·e	faux	vrai
Sydney	trentaines	client·e existant·e	vrai	vrai
Mariana	trentaines	nouveau·elle client·e	vrai	faux
Rasula	vingtaines	nouveau·elle client·e	vrai	vrai
Jillian	ados	client·e existant·e	faux	faux
Ariella	ados	nouveau·elle client·e	vrai	vrai
Isabela	trentaines	client·e existant·e	vrai	vrai

Sur la base d'un tel jeu de données, pourriez-vous écrire un classificateur qui prédit quel·les client·es sont les plus susceptibles d'acheter un jeu et leur envoyer une publicité ciblée ?

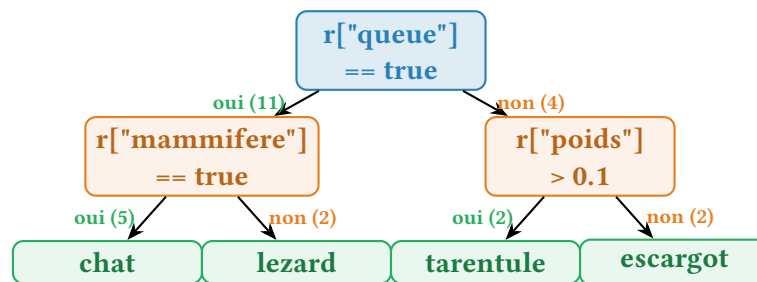
Filtrage de spam :

expediteur dans le carnet	expediteur deja vu	courriel de masse
vrai	vrai	vrai
vrai	faux	faux
faux	vrai	vrai
faux	faux	faux
faux	faux	vrai
vrai	vrai	faux
faux	faux	faux

Sur la base d'un tel jeu de données, pourriez-vous écrire un classificateur qui prédit si un courriel est du spam ou non ?

Annexe A : L'arbre de décision final en TikZ

L'arbre ci-dessous est un exemple d'arbre de décision à **trois niveaux** pour le jeu de données d'animaux. La racine divise sur `queue`, puis chaque branche se divise à nouveau. Les nœuds feuilles (en vert) montrent l'espèce prédite.



Légende :

- **Bleu** — nœud racine ;
- **Orange** — nœuds de décision intermédiaires ;
- **Vert** — nœuds feuilles (prédiction finale).

Notes pour l'enseignant·e

Cet arbre n'est qu'un exemple — vos élèves peuvent choisir des divisions différentes et obtenir un arbre tout aussi correct ! L'objectif est qu'ils justifient leur choix à partir des dot plots.

Annexe B : Fichiers du projet

Fichier	Description
Arbre-Decision-Starter.arr	Fichier de démarrage Pyret : charge <code>ai-library.arr</code> depuis Bootstrap + CSV français depuis GitHub.
construire-arbres-decision.tex	Document de cours (ce fichier)
dictionaries/training-fr.csv	15 animaux d'entraînement (colonnes <code>nom</code> , <code>espece</code> , <code>sexe</code> , <code>poids</code> , <code>queue</code> , <code>mammifere</code> , <code>nage</code>)
dictionaries/testing-fr.csv	15 animaux de test (mêmes colonnes)
images/	Diagrammes et dot plots de la leçon (12 PNG)

À noter : contrairement à l'original anglais qui utilise `load-spreadsheet` pour charger les données depuis Google Sheets, notre adaptation française charge les CSV depuis le dépôt GitHub du projet (`csv.csv-table-url`). Aucune dépendance Google n'est requise. La bibliothèque `ai-library.arr` de Bootstrap (qui fournit `classify`, `image-dot-plot`, etc.) est chargée via `use context url-file(...)` comme dans l'original.

À propos de ces supports

Ces supports ont été développés en partie grâce au soutien de la *National Science Foundation* (bourses 1042210, 1535276, 1648684, 1738598, 2031479 et 1501927).

Bootstrap par la Communauté Bootstrap est sous licence Creative Commons 4.0 Unported. Cette licence n'accorde pas la permission d'organiser des formations ou du développement professionnel.

Traduction et adaptation française réalisées en juillet 2026.

Source originale : <https://www.bootstrapworld.org/materials/fall2026/en-us/lessons/decision-trees-building/>